順序とカンニング

位置埋め込みと因果マスク — 自己注意の2大事件を解決する

rkskmt

前回の振り返り

- 自己注意が完成した：$\mathbf{softmax}(QK^\top/\sqrt{d})\,V$ — 行列積2回と softmax
- 予告した「2つの大問題」を、今日は**事件の再現**から始める

クイズ

自己注意に「ねこがねず」と「こねがねず」（前4文字を並べ替え、最後は同じ）を入れる。 **最後の文字の出力**は変わるか？

事件1：順番を替えても何も変わらない

Code

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SelfAttention(nn.Module):                          # 前回のクラスそのまま
    def __init__(self, n_embd, head_size):
        super().__init__()
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.key   = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)

    def forward(self, x):
        q, k, v = self.query(x), self.key(x), self.value(x)
        w = F.softmax(q @ k.T / k.shape[-1] ** 0.5, dim=-1)
        return w @ v

torch.manual_seed(0)
sa = SelfAttention(n_embd=8, head_size=4)
emb = nn.Embedding(10, 8)

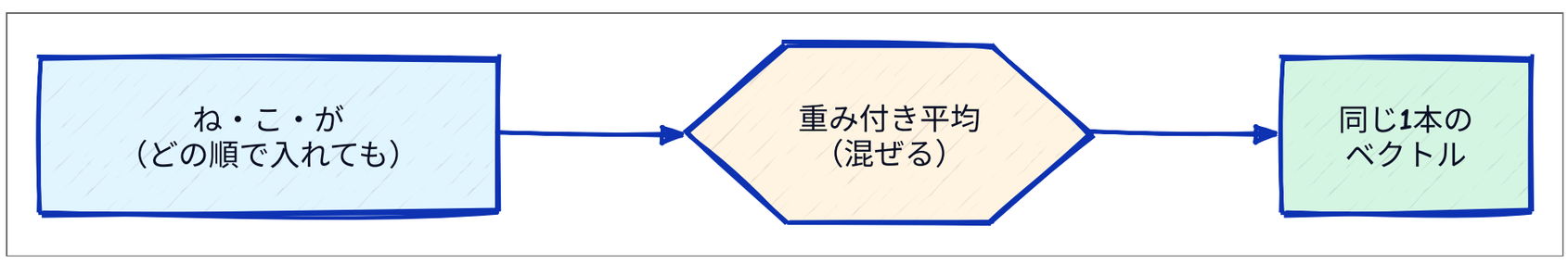
tokens   = torch.tensor([3, 1, 4, 1, 5])
shuffled = torch.tensor([1, 4, 1, 3, 5])              # 前4文字を並べ替え
diff = (sa(emb(tokens))[-1] - sa(emb(shuffled))[-1]).abs().max()
print(f"最後の文字の出力の差: {diff:.1e}")
```

Result

```
最後の文字の出力の差: 1.5e-08
```

- 差は**ほぼ0（丸め誤差だけ）**。「がねこ」と「ねこが」の区別が付いていない

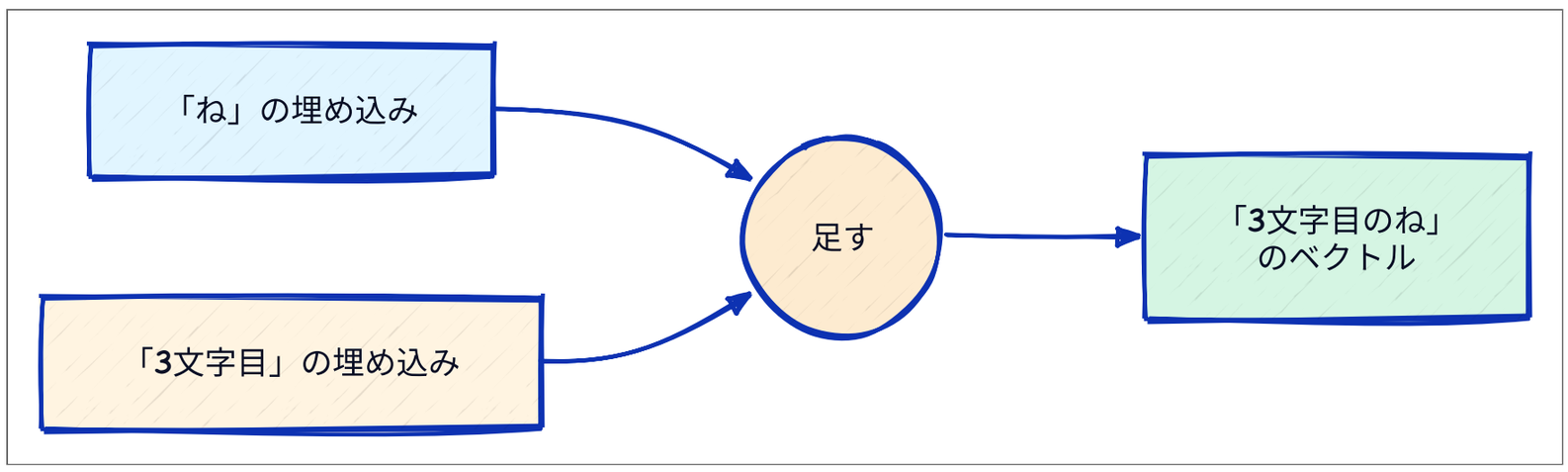
犯人は足し算 — 混ぜ物に順番はない



- 注意の出力は v の **重み付き平均** — つまり**足し算**。足す順番を替えても合計は変わらない
- 重みを決めるスコア $q \cdot k$ も1対1の内積で、「何番目か」はどこにも入っていない
- 自己注意は文を**文字の集合（袋）**として見ている — 「ねこが」も「がねこ」も同じ袋。日本語で順序が消えたら大惨事

解決1：位置にも埋め込みを配る

- 「何文字目か」という情報も、文字と同じように**埋め込みベクトル**にしてしまう
- 文字のベクトルに**足して**から自己注意に渡す



Code

```
pos = nn.Embedding(16, 8)                    # 位置番号 0〜15 の埋め込み表

def with_pos(t):
    return emb(t) + pos(torch.arange(len(t)))

diff = (sa(with_pos(tokens))[-1] - sa(with_pos(shuffled))[-1]).abs().max()
print(f"位置を足した後の差: {diff:.4f}")
```

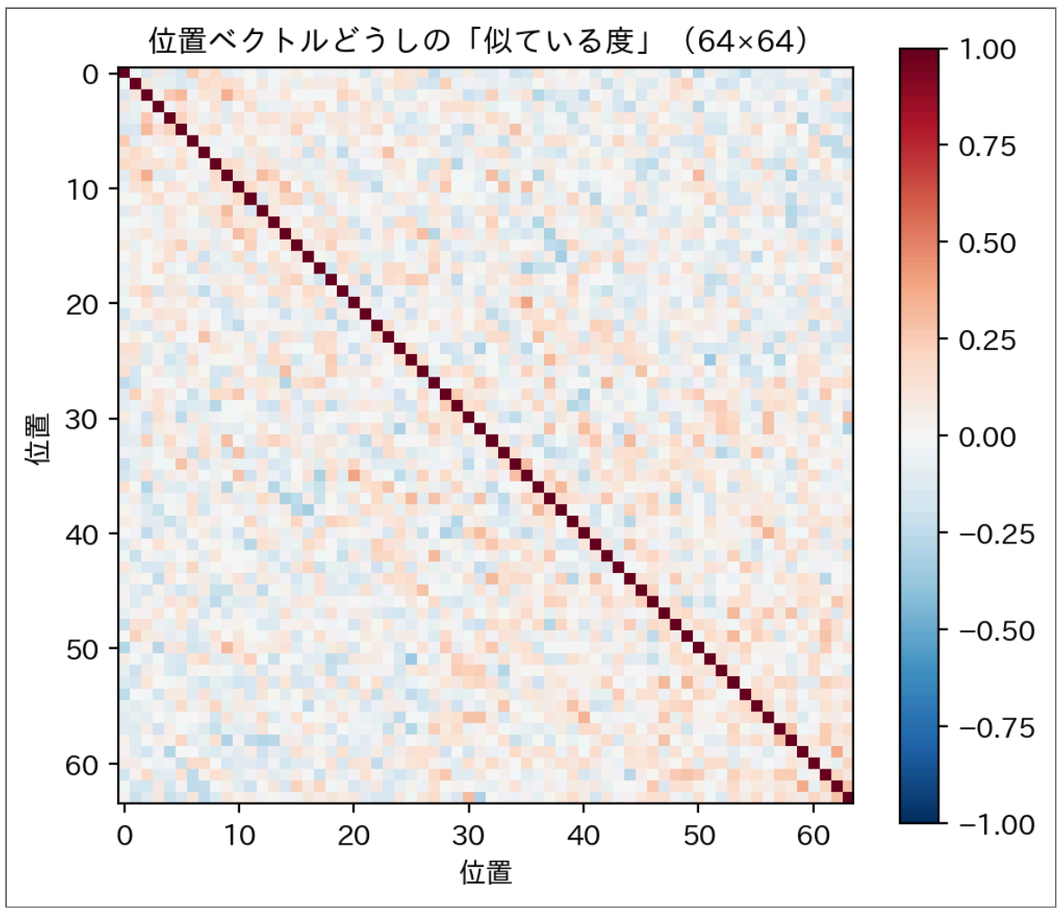
Result

```
位置を足した後の差: 0.1176
```

- 差が出た — 同じ「ね」でも**1文字目のね**と**4文字目のね**は別のベクトルになる
- この位置の表（**pos**）が**位置埋め込み**（positional embedding）。表も学習される — 「位置の使い方」まで勾配降下まかせ

学習された「位置の表」を覗く

- では、訓練を終えた表には何が入っているのか。完成品（最終回のモデル）から64本を取り出して比べる



- 赤いのは対角線（自分×自分）だけ — **どの位置とも似ていない、ほぼ直交の「住所札」64本**
- 隣どうしすら似せていない（隣接位置の平均類似度 0.08）。このモデルに必要だったのは「近さ」ではなく**区別**だった

事件2：訓練中の視線は未来を見ている

- 言語モデルの訓練は「各位置で**次の文字**を当てる」だった
- ところが前回作った**視線の表**（誰が誰をどれだけ見るかの $T \times T$）は**全部**埋まっている — 3文字目の行が5文字目も見ている

> **クイズ**
>
> 「次の文字を当てよ」というテスト中に、**答えの文字そのもの**が見えていたら何が起きる？ 訓練の成績と、生成（本番）の成績、それぞれどうなる？

- 訓練：正解を丸写しできるので**ほぼ満点**
- 本番の生成：未来の文字は**まだ存在しない**ので、丸写し戦略は白紙 — 大崩壊
- 「あやしい満点」、三たび。今回は過学習ではなく**カンニング**が原因

解決2：因果マスク — 未来の視線を封じる

- 視線の表の「未来」のマス（右上半分）を、softmax の**前**に $-\infty$ にする — これが**因果マスク**（causal mask）

Code

```
import torch
import torch.nn.functional as F

scores = torch.arange(1.0, 17.0).reshape(4, 4)      # 視線スコアの例
tril = torch.tril(torch.ones(4, 4))                 # 左下三角だけ1
masked = scores.masked_fill(tril == 0, float("-inf"))
print(masked)
print(F.softmax(masked, dim=-1).round(decimals=3))
```

Result

```
tensor([[ 1., -inf, -inf, -inf],
        [ 5.,  6., -inf, -inf],
        [ 9., 10., 11., -inf],
        [13., 14., 15., 16.]])
tensor([[1.0000, 0.0000, 0.0000, 0.0000],
        [0.2690, 0.7310, 0.0000, 0.0000],
        [0.0900, 0.2450, 0.6650, 0.0000],
        [0.0320, 0.0870, 0.2370, 0.6440]])
```

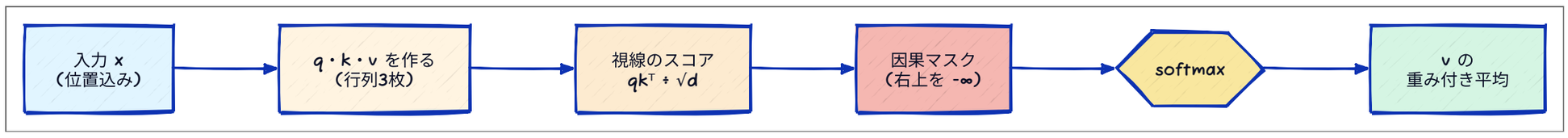
- softmax を通すと $e^{-\infty} = 0$ — 未来の重みが**きっかり0%**になる
- 1文字目は自分しか見えない、2文字目は2人まで…… 段々畑の視線になる
- 前回エクスプローラで見た「視線の表の**右上半分が空**」の正体がこれ — 訓練済みの本物には最初からこのマスクが入っていた

ⓘ なぜ 0 ではなく −∞ を入れる？

スコアに 0 を入れても $e^{0} = 1$ で**票が入ってしまう**（3人同点なら33%ずつ）。「見えない」を作れるのは $-\infty$ だけ。

注意ヘッド、完成

- 位置は入口で足すので、注意側の変更はマスクの2行だけ。この形を**ヘッド**（**head**）と呼ぶ



Code

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Head(nn.Module):
    """マスク付き自己注意（言語モデル用の完成形）"""

    def __init__(self, n_embd, head_size, block_size):
        super().__init__()
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.key   = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer("tril", torch.tril(torch.ones(block_size, block_size)))

    def forward(self, x):                              # x: (T, n_embd)
        T = x.shape[0]
        q, k, v = self.query(x), self.key(x), self.value(x)
        w = q @ k.T / k.shape[-1] ** 0.5
        w = w.masked_fill(self.tril[:T, :T] == 0, float("-inf"))   # 未来は見ない
        w = F.softmax(w, dim=-1)
        return w @ v

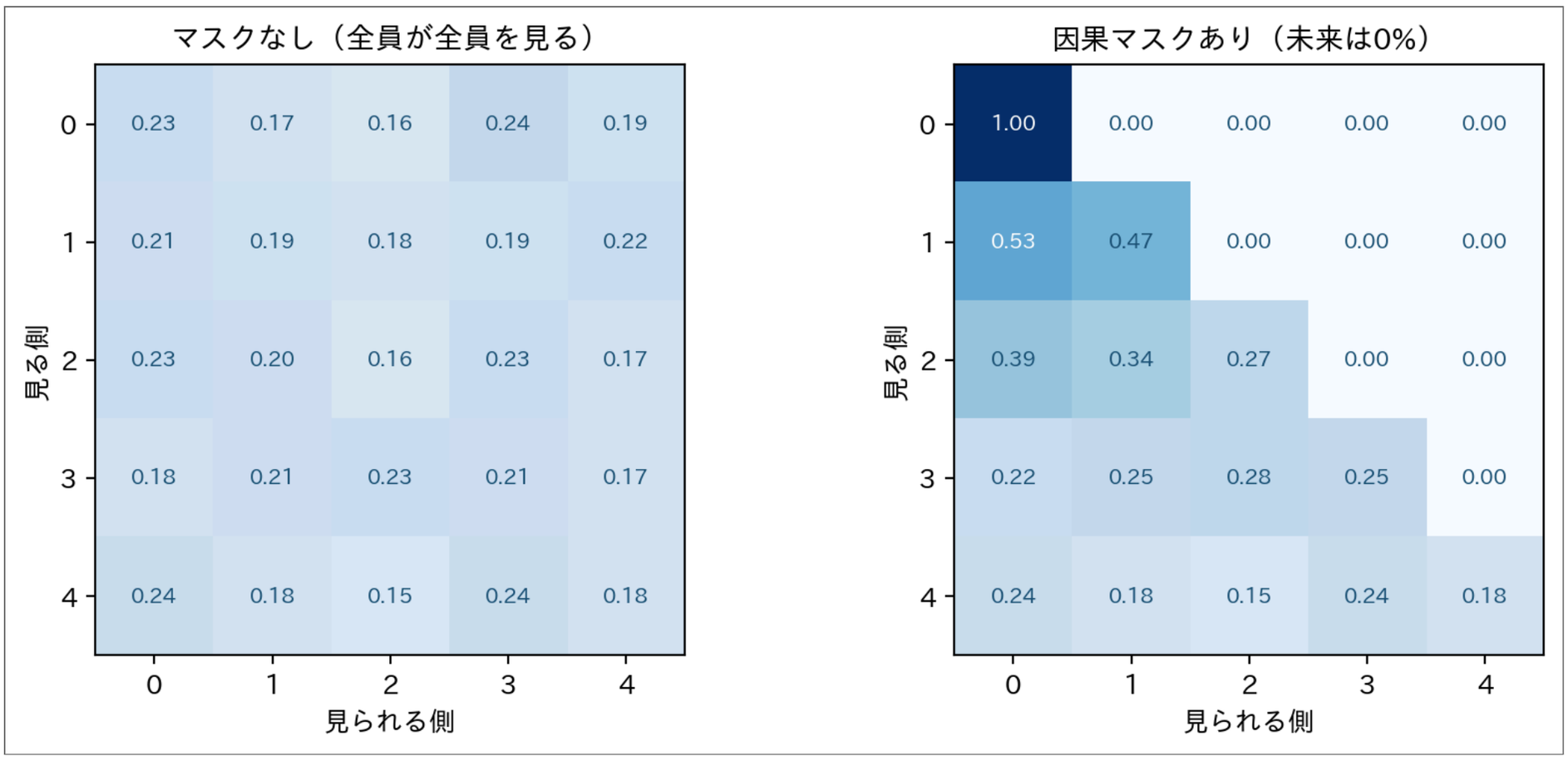
torch.manual_seed(0)
head = Head(n_embd=8, head_size=4, block_size=16)
print(head(torch.randn(5, 8)).shape)
```

Result

```
torch.Size([5, 4])
```

- **block_size** は「一度に見られる最大文字数」。**register_buffer** は「学習しない定数」の置き場

マスク前後の視線



- 右：どの行も**自分から左だけ**で合計100%。生成のときと同じ条件で訓練できる

Tweak 事件をもう一度起こす



- 本編のコードを組み合わせて実験（**`Head`** と **`with_pos`** を1つのファイルにまとめて実行）

1	**`masked_fill` の行をコメントアウト**	視線の表の右上が復活する（ヒートマップで確認）
2	**`float("-inf")` → `0.0`**	「見えない」が「同点の1票」に化ける。1行目の重みで確認
3	**`pos(...)` を足すのをやめる**	シャッフル実験の差がまた**ほぼ0**に戻る

ここで観察

- 3つとも「壊す実験」。**部品の役割は、外したときに一番よく分かる**
- マスクを外した瞬間、訓練成績だけが不当に良くなる — 数字が良くなる変更が改善とは限らない

まとめ

- 事件1：自己注意は**順序を知らない**（足し算だから）→ **位置埋め込み**を文字の埋め込みに足す
- 事件2：訓練中に**未来が見える**（カンニング）→ **因果マスク**で右上を $-\infty$ に
- $e^{-\infty} = 0$ が「見えない」を作る。0 ではダメ（票が入る）
- 言語モデル用の注意ヘッドが完成 — **Head** クラスはこの先そのまま使う

現在の設計図　文字＋位置 → **因果マスクつき自己注意** → 文脈つきベクトル

> **ⓘ 次回予告**
>
> ヘッド1本では視線が1系統しかない — 「直前を見る」と「カッコを見る」は両立できない。
> 次回、ヘッドを**束ねて**視線を複線化し、集めた情報に**考える場所**を付ける。

練習問題

以下の問題を**Pythonコード**で解け。

問題1：$-\infty$ と 0 の違い

スコア $(-\infty, 0, 0)$ と $(0, 0, 0)$ をそれぞれ softmax に通し、結果の違いを表示せよ。「マスクに 0 を使ってはいけない理由」を出力を根拠に1行で述べよ。

問題2：コンテキスト長の正体

`Head(n_embd=8, head_size=4, block_size=16)` に **17文字**（`torch.randn(17, 8)`）を入れると何が起きるか。予想してから実行し、エラーの原因を `tril` の形から説明せよ。

1. 実行してエラーメッセージを読む
2. ChatGPT の仕様表にある「コンテキスト長（一度に扱える長さ）」が何によって決まっていたか、今日の部品で説明

解答例

▶ 解答例を見る

Result

```
tensor([0.0000, 0.5000, 0.5000])
tensor([0.3333, 0.3333, 0.3333])
RuntimeError: The size of tensor a (16) must match the size of tensor b (1 ...
```